

EXAM REVIEW!!!

EXAM REVIEW!

- Wednesday, October 15th during class
- Covers everything through sets, dictionaries, and unit testing
- Pencil & paper, so be ready...
- Practice exams on website

YOUR REQUESTS

...

PRACTICE

Penn is developing an app for its dining halls. It uses a dictionary that maps names to lists of diets that the dining hall supports, e.g.

```
{  
    "Commons" : ["Vegetarian", "Vegan", "Kosher", "Halal"],  
    "Hillel" : ["Kosher", "Vegetarian"],  
    "Harry's Kitchen" : []  
}
```

You will write a function that returns a set containing the names of all dining halls that support a given diet, i.e.

```
def find_halls_with_diet(halls: dict[str, list[str]], diet: str) -> set[str]:  
    ...
```

1. Write two unit tests that verify the correct behavior of `find_halls_with_diet()`.
2. Implement `find_halls_with_diet()`.

ANSWERS

```
import solution # we'll give you this part

def test_no_halls_matching(self):
    diet = "dubious"
    halls = {"Harry's Kitchen" : [], "Commons" : ["Vegan", "Halal"]}
    expected = set()
    actual = solution.find_halls_with_diet(diet, halls)
    self.assertEqual(expected, actual)

def test_all_halls_matching(self):
    diet = "Vegan"
    halls = {"Hill" : ["Vegan"], "Commons" : ["Vegan", "Halal"]}
    expected = {"Hill", "Commons"}
    actual = solution.find_halls_with_diet(diet, halls)
    self.assertEqual(expected, actual)
```

ANSWERS

```
def find_halls_with_diet(halls: dict[str, list[str]], diet: str) -> set[str]:  
    matching_halls = set()  
    for name, diets in halls.items():  
        if diet in diets:  
            matching_halls.add(name)  
    return matching_halls
```

OR...

```
def find_halls_with_diet(halls: dict[str, list[str]], diet: str) -> set[str]:  
    matching_halls = set()  
    for name in halls:  
        if diet in halls[name]:  
            matching_halls.add(name)  
    return matching_halls
```

DEBUGGING

All dining halls should support a vegetarian diet. But a vegan diet is always also vegetarian. Update a dictionary so that all dining halls include "Vegetarian", but *do not add "Vegetarian"* if they already include "Vegan" or "Vegetarian". Find the three lines with bugs in the code below. List the line numbers, describe the bug, and rewrite the part of the line that requires a fix.

```
def eat_your_veggies(halls: dict[str, list[str]]):  
    for name in halls:  
        diets = halls[name]  
        found_match = False  
        for diet in diets:  
            if diet == "Vegetarian" and diet == "Vegan":  
                found_match == True  
        if not found_match:  
            diets += "Vegetarian"
```

DEBUGGING (ANSWER)

All dining halls should support a vegetarian diet. But a vegan diet is always also vegetarian. Update a dictionary so that all dining halls include "Vegetarian", but *do not add "Vegetarian"* if they already include "Vegan" or "Vegetarian". Find the three lines with bugs in the code below. List the line numbers, describe the bug, and rewrite the part of the line that requires a fix.

```
def eat_your_veggies(halls: dict[str, list[str]]):  
    for name in halls:  
        diets = halls[name]  
        found_match = False  
        for diet in diets:  
            if diet == "Vegetarian" or diet == "Vegan":  
                found_match = True  
        if not found_match:  
            diets.append("Vegetarian")
```

FITB

A local event registration system assigns each participant a unique numeric ID when they sign up. For logistical reasons, only participants with even-numbered IDs greater than 15 are eligible for an invitation to a special workshop. Instead of manually searching through the list of IDs, the team decides to write a Python program to automate this filtering process.

The team has started writing the program but needs your help fixing some key parts. Help them write a function that takes in a list of numbers, filters out all IDs that are not even or are less than or equal to 15, converts the remaining IDs to strings, and returns them as a single, comma-separated string.

e.g. `filter_and_format([20, 15, 4, 6, 17, 42]) -> "20, 42"`

Help them write a function that takes in a list of numbers, filters out all IDs that are not even or are less than or equal to 15, converts the remaining IDs to strings, and returns them as a single, comma-separated string.

```
def filter_and_format(numbers):  
    result = []  
    for num in numbers:  
        if num % 2 == 0 and num > __BLANK_1__:  
            result.append(str(__BLANK_2__))  
    final_string = ""  
    for i in range(len(result)):  
        final_string += result[i]  
        if i != __BLANK_3__: # only add a comma if it's not the last item in the list  
            final_string += ", "  
    return final_string
```